

XVG Format Specification (v1.0)

Introduction

XVG (eXtended Vector Graphics) is a compact binary format for storing and rendering scalable vector graphics. It is designed for situations where size, speed and quality matter: icons, animations, UI elements and complex scenes. By representing geometry, colour, optional assets and advanced features in a fixed binary layout, XVG files are smaller than equivalent SVGs and quicker to parse, while remaining self-contained (no external resources) and extensible for future needs.

File structure

An XVG file is a sequence of segments concatenated in a well-defined order:

1. **Header** – Identifies the file, provides canvas dimensions, animation parameters and feature flags. The header length is encoded to allow future extensions.
2. **Metadata** – A gzip-compressed JSON document containing human-readable information such as palette definitions, viewBox, frame labels, creation metadata and resolution hints. The length of this segment precedes the compressed data.
3. **Frame table** – For animated files, this table lists the byte offset of each frame's command stream and optionally a per-frame duration when variable frame rates are used.
4. **Command streams** – A series of drawing commands encoded as opcodes. Each command begins with a 4-byte little-endian length that counts the opcode and parameter bytes, allowing unknown commands to be skipped.
5. **Embedded assets** – A list of binary resources such as images, audio, fonts and shaders. Each asset record includes type, name, size and payload. Assets are optional; simple files may omit this segment.
6. **Optional sections** – Additional data enabled via feature flags: signed distance field weights, a collaborative editing log, a table of WGSL shaders, or transformation matrices for 3-D extrusion.
7. **Section offset table** – An optional lookup table for random access. It maps section identifiers to absolute byte offsets.
8. **Checksum and sentinel** – A 32-bit CRC checksum of the entire file (excluding the checksum itself and the sentinel) followed by the seven-byte ASCII string `XVGEOF` to mark the end of the file.

Header

The header starts with the magic bytes `XVG\x02`. The next 2 bytes (`uint16`, big-endian) specify the total length of the header. This allows new fields to be added without breaking existing parsers. The remaining fields are:

- **Canvas width** (`uint16`, little-endian) – The width in pixels of the default coordinate space.
- **Canvas height** (`uint16`, little-endian) – The height in pixels of the default coordinate space.

- **Legacy flags** (`uint16`, little-endian) – Bits controlling legacy behaviour (animated, has assets, alpha channel, looping). These are retained for backward compatibility.
- **Frame count** (`uint32`, little-endian) – Number of animation frames. A value of 1 indicates a static image.
- **Frame rate** (`float32`, big-endian) – Frames per second. A value of 0.0 indicates variable frame rate; durations are then read from the frame table.
- **Feature flags** (`uint32`, big-endian) – A bitfield enabling optional sections. Bits 4–7 correspond to the signed distance field (bit 4), collaborative log (bit 5), shader table (bit 6) and 3-D scene (bit 7). Bits 0–3 mirror the legacy flags; bits above 7 are reserved and must be zero.

Parsers should ignore unknown bits but must ensure reserved bits are zero. Additional header fields may be added in future versions after the feature flags; the header length must be adjusted accordingly.

Metadata

Immediately after the header, a 32-bit little-endian length indicates the size of the compressed metadata. This segment contains a gzip-compressed JSON object. The object is optional; parsers may skip decompressing it entirely. Fields commonly defined include:

- **palette**: An array of colour values (e.g. `"#FF0000"`) used by indexed colour commands.
- **viewBox**: A string such as `"0 0 100 100"` defining the origin and extents of the coordinate system.
- **frames**: An array of objects with labels and durations used when variable frame rates are enabled.
- **meta**: User metadata such as author, creation time, license and generator information.
- **resolutions** and **densities**: Arrays and maps suggesting canonical output sizes for multi-resolution rendering.
- **shaderFallback** and **shaderFallbacks**: Colour or pattern values to use when shaders are unsupported.

Additional keys may be added without affecting compatibility. Tools should preserve unrecognised keys when reading and writing metadata.

Frame table

If the file contains multiple frames (frame count > 1 or the animated flag is set), a frame table follows the metadata. Each entry consists of:

- **Frame offset** (`uint32`, little-endian) – The byte offset of the first command for that frame relative to the start of the file.
- **Frame duration** (`float32`, big-endian) – Optional. Only meaningful when the header's frame rate is 0.0. If constant frame rate is used, implementations may omit durations altogether.

The number of entries in the frame table must match the frame count. For static images, the table may be omitted entirely.

Command stream

The command stream encodes drawing instructions. Each command begins with a 4-byte little-endian length indicating the total size of the command (opcode plus parameters). Commands must not overlap. Unknown opcodes can therefore be skipped by reading their length and advancing the file pointer.

Parameters use big-endian encoding unless specified. The core set of opcodes is summarised below:

Path construction

Opcode	Name	Parameters	Meaning
0x01	MoveTo	<code>x, f32</code> , <code>y, f32</code>	Start a new subpath at (x, y).
0x02	LineTo	<code>x, f32</code> , <code>y, f32</code>	Draw a straight line to (x, y).
0x03	CurveTo	<code>x1, f32</code> , <code>y1, f32</code> , <code>x2, f32</code> , <code>y2, f32</code> , <code>x, f32</code> , <code>y, f32</code>	Draw a cubic Bézier curve.
0x04	ClosePath	<code>fillRule, u8</code>	Close the current path; optionally fill using the rule.

Colour and stroke

Opcode	Name	Parameters	Meaning
0x10	SetFillColour	<code>R, u8</code> , <code>G, u8</code> , <code>B, u8</code> , <code>A, u8</code>	Set the fill colour directly.
0x11	SetStrokeColour	<code>R, u8</code> , <code>G, u8</code> , <code>B, u8</code> , <code>A, u8</code> , <code>width, f32</code>	Set the stroke colour and stroke width.
0x12	SetFillIndexed	<code>index, u8</code>	Select a fill colour from the palette.
0x13	SetStrokeIndexed	<code>index, u8</code>	Select a stroke colour from the palette.

Transformations and guides

Opcode	Name	Parameters	Meaning
0x40	PushTransform	<code>matrix, 9×f32</code>	Multiply the current transform by a 3×3 matrix.
0x41	PopTransform	–	Restore the previous transform.
0x50	LineGuide	<code>x1, f32</code> , <code>y1, f32</code> , <code>x2, f32</code> , <code>y2, f32</code>	Draw a guide line (editor only).

Opcode	Name	Parameters	Meaning
0x51	RectGuide	<code>x, f32</code> , <code>y, f32</code> , <code>w, f32</code> , <code>h, f32</code> , <code>angle, f32</code>	Draw a guide rectangle (editor only).

Advanced features

Opcode	Name	Purpose
0x60	DefineSDF	Start a signed distance field definition.
0x61	DrawSDF	Render the previously defined SDF shape.
0x70	BindFillShader	Bind a WGSL shader to the fill operation.
0x71	BindStrokeShader	Bind a WGSL shader to the stroke operation.
0x80	PushMatrix	Push a 3-D transformation matrix onto the stack.
0x81	PopMatrix	Pop the last 3-D transformation matrix.
0x82	Extrude	Extrude the current path into a 3-D mesh.

Reserved opcodes ($\geq 0x90$) may be defined in future revisions. Decoders should skip unknown opcodes using their length prefix.

Embedded assets

An asset record begins with three one-byte fields: a type identifier, a flags byte and the length of the asset name (UTF-8). The name string follows immediately. Next are two 32-bit little-endian lengths: the size of the (compressed) payload and, if bit 0 of the flags byte is set, the size of the uncompressed payload. The data itself follows. Asset type values below 0x10 are reserved; values 0x10 and above may be used by applications. Common built-in types are:

- **0x01** – Image (PNG, JPEG, WebP)
- **0x02** – Audio (MP3, OGG, WAV)
- **0x03** – Font (TTF, OTF, WOFF2)
- **0x04** – Generic binary blob
- **0x05** – WGSL shader program
- **0x06** – SDF weight data

Parsers must skip unknown asset types by reading the declared size.

Optional sections

Signed distance fields (SDF)

When the **InfiniteResSDF** flag is set (bit 4 of the feature flags), a section containing neural network weights follows the assets. It begins with a 32-bit little-endian count of weight blocks and a 16-bit big-endian grid

hint. The weights are stored as half-precision floats compressed with gzip. SDF opcodes (`DefineSDF`, `DrawSDF`) reference these weight blocks by index. SDF allows mathematically perfect rendering at any zoom level.

Collaborative editing (CRDT)

If the **LiveCRDT** flag is set (bit 5), the CRDT log section stores editing operations for collaborative workflows. The section begins with a 32-bit little-endian log length and then a sequence of entries. Each entry consists of an 8-bit length, a 64-bit big-endian Lamport timestamp, a 16-bit little-endian author ID and a sequence of standard drawing commands. Offline or single-user applications may ignore this section entirely.

Shader table

When **ShaderStrokes** is enabled (bit 6), a shader table stores WGSL source code used to render custom strokes and fills. Each shader is an asset (type 0x05) and may have a fallback colour defined in the metadata. The opcodes `BindFillShader` and `BindStrokeShader` select shaders by ID. Shaders are executed at fragment time with UV coordinates, base colour and time, returning a colour.

3-D scene graph

If the **Scene3D** flag is set (bit 7), a section contains a sequence of 4×4 matrices (32-bit floats, row-major) that form a stack of 3-D transformations. The section begins with an 8-bit depth indicating how many matrices follow. Commands `PushMatrix`, `PopMatrix` and `Extrude` use and manipulate this stack. Applications not supporting 3-D may ignore these opcodes.

Section offset table

For random access to optional sections, a section offset table may appear after the optional data. It starts with a single byte indicating the number of entries. Each entry consists of a 1-byte section ID and a 4-byte big-endian absolute offset. Section IDs 6–9 correspond to the SDF, CRDT, Scene3D and Shader sections, respectively. Values above 9 are reserved.

Integrity blocks

To detect corruption in the command stream, implementations may insert periodic integrity blocks. These consist of the special opcode `0xFF` followed by a 4-byte big-endian CRC-32 of the preceding 4 KB of command data. Decoders should verify these values if present.

Shader fallbacks and resolution hints

The metadata may define a single `shaderFallback` colour or a map `shaderFallbacks` from shader names to fallback colours or patterns. Clients that do not support WGSL should use these values. Similarly, the `resolutions` array and `densities` map suggest recommended output sizes; renderers may choose the closest supported resolution.

Runtime API

All XVG runtimes must expose two functions:

1. `render(width, height, options)` – Renders the file into a bitmap, mesh, texture or vector output. The `width` and `height` specify the output resolution. Optional parameters include:
2. `target` – The desired output type (`bitmap`, `mesh`, `texture`, `svg`).
3. `device` – A GPU device or context for hardware acceleration.
4. `frame` – A 0-based index selecting an animation frame.
5. `background` – A tuple defining the background colour.

The function returns a structure containing the rendered data appropriate to the target.

1. `extract(format, options)` – Extracts data or converts the XVG file into another format. The first argument specifies the desired output format (e.g. `"svg"`, `"png"`, `"ico"`, `"gltf"`, `"woff2"`, `"mp3"`). Format-specific options may be provided (e.g. DPI for PNG, quality for audio). This function is intended for build pipelines; runtimes do not perform conversions during rendering.

MIME type and extension

XVG files use the MIME type `model/xvg` and the extension `.xvg`. They can be detected by the magic bytes `XVG\x02` at the start of the file. All valid files end with the sentinel `XVGEOF` immediately after the CRC-32 checksum.

Security considerations

XVG is designed to be secure by default. Key considerations include:

1. **No scripting** – Unlike SVG, XVG does not allow embedded scripts. This eliminates script injection risks.
2. **No external references** – All assets are embedded in the file; network requests are not permitted. Parsers must reject files that attempt to load external resources.
3. **Integrity checks** – A global CRC-32 checksum covers the entire file. Additional integrity blocks may protect command streams. Implementations must verify the checksum before processing.
4. **Resource limits** – Decoders should impose limits: maximum file size (e.g. 2 GB), canvas dimensions (e.g. 65 535×65 535), frame counts (e.g. 4 × 109) and asset counts (e.g. 255 per type). Files exceeding these limits should be rejected.
5. **Graceful failure** – Parsers must skip unknown opcodes and sections, reporting non-fatal errors where possible. Malformed data should not crash the application.

Implementation guidance

Parsing

Implementations should:

1. Read the header, verifying the magic bytes and ensuring the header length does not exceed the file size.
2. Parse canvas dimensions, frame count, frame rate and feature flags. Validate that reserved bits are zero.
3. Decompress and parse the metadata if needed.
4. Read the frame table if the file is animated. Use the frame rate field to determine whether durations are required.
5. Iterate over commands for each frame. For each command, read the length, opcode and parameters. Unknown opcodes must be skipped using the length field.
6. Locate and process optional sections based on the feature flags. Use the section offset table if present.
7. Verify the CRC-32 checksum and any command integrity blocks.

Rendering

Renderers should:

1. Support at minimum the basic drawing opcodes (`MoveTo`, `LineTo`, `CurveTo`, `ClosePath`) and colour opcodes. Unsupported opcodes should be ignored with a warning.
2. Maintain a transformation stack for `PushTransform` and `PopTransform`. Apply the current transform to all subsequent path operations.
3. Honour transparent backgrounds and apply the specified background colour only if a background fill is absent.
4. Implement optional features conditionally: ignore SDF definitions when the `InfiniteResSDF` flag is not set; ignore CRDT entries when live collaboration is disabled; ignore shaders when shader support is absent.
5. Provide sensible fallbacks (e.g. use the `shaderFallback` colour when ignoring shaders).

Authoring tools

Tools that generate or manipulate XVG files should:

1. Preserve existing metadata and unknown sections when editing.
2. Update the frame table and command offsets whenever the command stream changes.
3. Compress metadata and weights using gzip to reduce file size.
4. Compute and write the correct CRC-32 checksum and sentinel at the end of the file.
5. Validate input files and report detailed error messages for malformed files.

Extension mechanism

XVG reserves feature flag bits beyond bit 7, section identifiers beyond 9, opcode values $\geq 0x90$ and asset type identifiers $\geq 0x10$ for future extensions. Authors of new features must allocate a feature flag, a section ID (if a new section is added), one or more opcodes and optional asset types. Extensions must specify how unknown flags and opcodes are to be handled and should remain backwards-compatible where possible. Planned extension areas include animation easing functions, advanced text layout, physics simulation, AR/VR spatial metadata and provenance via on-chain records.

This specification defines XVG version 1.0. Implementers are encouraged to provide feedback and contribute to future revisions.
